BENGALURU NORTH UNIVERSITY

PROJECT REPORT

STUDENT PERFORMANCE ANALYTICS SYSTEM

A Python-based analysis of student marks and attendance

Submitted by	**Umera Sulthana S**
USN	U19GA24SO102
Course	Bachelor of Computer Applications (BCA)
Semester	5th Semester
Project Guide	Chandini Shetty

ABSTRACT

The Student Performance Analytics System is a Python project that analyses student marks and attendance from a CSV dataset. It calculates total marks, averages, grades, pass/fail results, rankings and subject-wise performance. The system also checks the input data, identifies students with attendance below 75%, and saves processed results as CSV reports. Pandas is used to read and organise the data, NumPy supports numerical calculations, and Matplotlib creates optional charts. The project turns a table of student records into a clear summary that can help teachers and students understand academic performance. The included dataset has 25 sample records and is used for demonstration, not as official student data.

KEYWORDS: Python, Pandas, NumPy, Matplotlib, Data Analysis, Student Performance

1. INTRODUCTION

Student marks and attendance are often stored in tables, but reviewing every record manually can take time. This project uses Python to process the records and present the results in a consistent format. It is designed as a small academic analytics tool that demonstrates data handling, calculations, validation and reporting.

1.1 Problem Statement

Manual calculation of totals, averages, grades and rankings can lead to repeated work and mistakes. It may also be difficult to compare subjects or quickly identify students who need support. A simple automated system can reduce this effort and make the results easier to review.

1.2 Objectives

• Read student marks and attendance from a CSV file.

• Validate required columns and check for missing, duplicate or out-of-range data.

• Calculate totals, averages, grades and pass/fail status.

• Find class and subject averages, toppers and student rankings.

• Flag attendance below 75% and allow a student to be searched by ID.

• Export the processed results for later review.

1.3 Scope

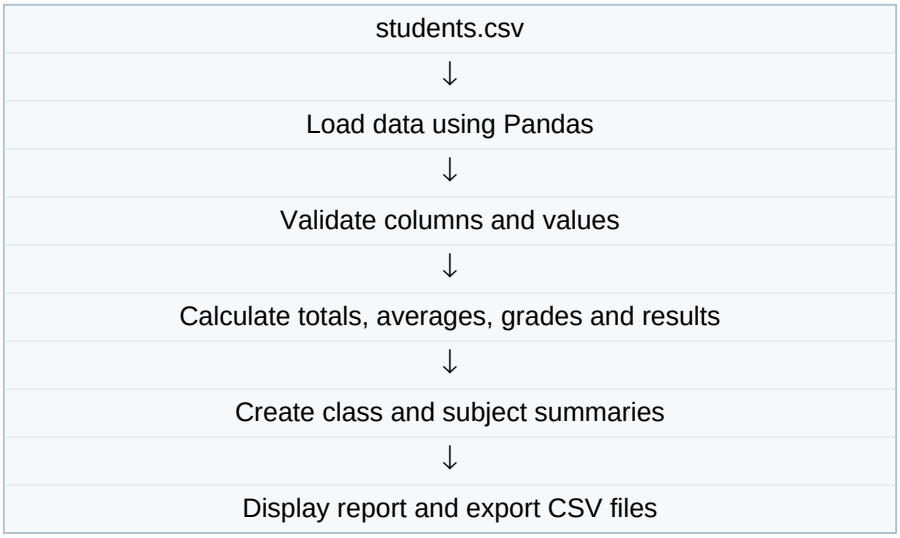
The project is intended for learning and small-scale academic analysis. It works with a local CSV dataset and produces printed summaries and CSV files. It does not include a web interface, database server, login system or predictive machine-learning model.

2. TOOLS AND SYSTEM REQUIREMENTS

Component	Requirement / Use
Programming language	Python 3.10 or newer
Libraries	Pandas, NumPy; Matplotlib for optional charts
Data format	CSV file (students.csv)
Hardware	Computer with basic processing and storage; keyboard and display
Operating system	Windows, Linux or macOS with Python installed

3. SYSTEM DESIGN AND METHODOLOGY

The system follows a simple sequence: load the dataset, validate it, process the marks, create summaries, display the report and save output files. Keeping the work in separate functions makes the code easier to test and maintain.



3.1 Main Modules

• **main.py:** Starts the program, loads the dataset, calls the processing functions and saves the output reports.

• **functions.py:** Contains reusable functions for validation, grade assignment, pass/fail decisions, summaries, rankings and student search.

• **students.csv:** Stores 25 sample student records, including five subject marks and attendance.

• **generate_charts.py:** Creates optional charts for subject averages, pass/fail counts and attendance.

3.2 Processing Rules

The system calculates total marks across five subjects and divides the total by five to obtain the average. Grades are assigned using the project thresholds: A+ (90 and above), A (80–89.99), B (70–79.99), C (60–69.99), D (50–59.99) and F (below 50). A student passes only if each subject mark is at least 35 and the overall average is at least 40. Attendance below 75% is flagged separately and does not change the pass/fail result.

4. IMPLEMENTATION AND TESTING

The project uses pandas.read_csv() to read the CSV file. It checks that the expected columns are present and that the marks and attendance values are valid. NumPy and Pandas are then used to calculate totals, averages and summaries. Conditional statements assign grades and results, while reusable functions keep the calculations organised.

4.1 Main Outputs

Output file	Purpose
student_results.csv	Student-wise totals, averages, grades, result and ranking information.
class_summary.csv	Overall count, pass/fail totals, class average and performance indicators.
subject_analysis.csv	Average marks for each subject.
PNG charts	Optional visual summaries of subjects, pass/fail and attendance.

4.2 Testing

Test	Expected behaviour	Observed / verified
Load valid dataset	Read student rows and columns	25 records loaded
Calculate class summary	Count pass/fail and compute average	22 passed, 3 failed
Subject analysis	Calculate average for each subject	Five subject averages generated
Export reports	Write summary CSV files	Three CSV reports present
Invalid/missing data	Show validation error	Validation implemented; test with edited data before submission

5. RESULTS AND DISCUSSION

The included sample run contains 25 student records. The verified class average is 70.58. In this sample, 22 students pass and 3 fail under the defined project rules. The highest average is 96.00 and the lowest is 36.40. Computer Networks has the highest subject average at 71.92, while Python has the lowest at 69.72. These values describe the demonstration dataset only and should not be treated as real institutional results.

6. CONCLUSION AND FUTURE ENHANCEMENTS

The Student Performance Analytics System shows how Python can turn a simple CSV file into useful academic summaries. It automates repeated calculations, applies consistent grading rules, compares subject performance and creates exportable reports. The project also demonstrates data validation and the use of functions, loops, conditions, Pandas and NumPy. Since the dataset contains sample records, the results are intended only to demonstrate the program.

Future Enhancements

• Add a graphical user interface for easier use.

• Store records in a database instead of a local CSV file.

• Allow authorised users to update student records.

• Add filters and interactive charts for subject and attendance analysis.

• If suitable real data is available, explore responsible prediction of students who may need support.

7. REFERENCES

1. Python Software Foundation, Python Documentation.

2. pandas documentation, data analysis and CSV handling.

3. NumPy documentation, numerical computing.

4. Matplotlib documentation, plotting and charts.

STUDENT DECLARATION

I declare that this project report has been prepared for academic purposes based on the Student Performance Analytics System. The program outputs described in this report are from the sample dataset included with the project. I will review the report and confirm that the details accurately represent my work before submission.

Student Signature: _____

Name: Umera Sulthana S

Guide: Chandini Shetty